CSCI3235 Midterm I Report

Tabular SARSA on slippery FrozenLake

Student: **Zhaxybayev Aibar**, ID: **23B030353** Date: 26 October 2026

1. Problem and RL suitability

The task is the slippery FrozenLake 4x4 environment from Gymnasium. An agent starts in the top-left cell of a frozen lake and must reach the goal in the bottom-right cell without falling into one of four holes. The decision-maker is the agent, and the desired outcome is to reach the goal in as many episodes as possible.

Why the decisions are sequential. Every move changes the agent's position, and the position decides which moves are safe next. The only reward (+1) arrives at the goal, so early moves are judged only by what they lead to later. The ice is slippery: the agent moves in the intended direction with probability 1/3 and in each of the two perpendicular directions with probability 1/3. A good plan must therefore keep the agent away from cells next to holes, even if that path looks longer.

What must be learned. The agent is not told the transition probabilities or where the reward is. It learns the value of each action in each cell from the episodes it experiences.

Non-RL alternative. Dynamic programming (value iteration) on a known model, or a fixed shortest-path rule. If the full model is known and small, value iteration gives the exact optimal policy without trial and error, so it would be preferable. A shortest-path rule is enough when the ice is not slippery. RL becomes useful when the model is unknown or hard to write down, because it only needs the ability to interact with the environment.

Personal connection. I do not have a direct application for this project yet. My connection is a general interest in how an agent can learn from trial and error, as in games and robot navigation.

2. MDP formulation

The environment is a small episodic MDP. The map (S = start, F = frozen, H = hole, G = goal) and the cell numbering are:

```
S F F F        0   1   2   3
F H F H        4   5   6   7
F F F H        8   9  10  11
H F F G       12  13  14  15
```

Element	Definition
States	16 cells, numbered 0-15 row by row. The agent observes the cell index.
Actions	4 actions: 0 = left, 1 = down, 2 = right, 3 = up.
Reward	+1 when the goal (state 15) is reached, 0 on every other step.
Transitions	Stochastic (slippery ice). The agent moves in the intended direction with probability 1/3 and in each perpendicular direction with probability 1/3. A move into a wall leaves the agent in the same cell.
Start	State 0, always.
Terminal states	Holes (5, 7, 11, 12) and the goal (15). An episode also ends after 100 steps (time limit of the Gymnasium environment).

Element	Definition
Discount	gamma = 0.99. The reward comes only at the end, so a high gamma keeps the goal valuable from far away, while gamma < 1 still slightly prefers shorter paths.

Type of task. The environment is stochastic and the task is episodic. The policy is stochastic during training (epsilon-greedy) and deterministic during evaluation (greedy).

Markov assumption. The cell index is a Markov state: it alone determines the distribution of the next cell and the reward, and the history adds nothing. The only approximation is the 100-step time limit, because the step counter is not part of the state, so the agent cannot know how much time remains. This matters little here, since most episodes end in a hole or the goal long before 100 steps.

One complete step (transition example). Take state 0 and action 2 (right). Because of slipping, the next state is one of three cells, each with probability 1/3, and the reward is 0 in all cases:

State	Action	Actual move	Reward	Next state
0	2 (right)	right (as intended)	0	1
0	2 (right)	down (slip)	0	4
0	2 (right)	up (slip, wall)	0	0

One step observed in my own run: s = **0**, a = **2**, r = **0**, s' = **1** .

3. Method and update rule

I use tabular SARSA, an on-policy temporal-difference control method. The agent keeps a table Q of size 16 x 4, initialised to zeros. Q(s, a) estimates the expected discounted return if the agent takes action a in state s and then keeps following its current epsilon-greedy policy.

After every step the agent observes the tuple (s, a, r, s', a') and applies the update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma\, Q(s', a') - Q(s, a) \right]$$

Term	Meaning in my own words
Q(s, a)	Current estimate of how good it is to take action a in cell s. This is the quantity being learned.
α = 0.1	Step size: how far one new observation moves the old estimate. A small α averages out the randomness of slipping; a large α reacts quickly but is noisy.
r	The reward just received (1 at the goal, otherwise 0).
γ = 0.99	How much the value of the next step counts compared with the immediate reward.
Q(s', a')	The estimate for the next cell and the action the agent will actually take there. It is set to 0 if s' is terminal (hole or goal), because nothing follows.
r + γ Q(s', a')	The target: a better guess of the value, built from one real step plus the current estimate of the future.

Term	Meaning in my own words
target − Q(s, a)	The error between the new guess and the old estimate. The update moves Q a fraction α of the way to the target.

What is estimated and how the policy uses it. The table estimates action values of the policy that is actually followed, including its random exploratory moves. This is why SARSA is called on-policy: a' is chosen by the same epsilon-greedy rule that chooses a. The policy uses Q like this: with probability ε the agent picks a random action (exploration), otherwise it picks the action with the largest Q(s, ·) in the current cell, breaking ties at random (exploitation). Ties matter at the start, when all values are zero.

Training and evaluation. During training the agent updates Q after each step with the epsilon-greedy policy. For evaluation, Q is frozen and the agent follows the greedy policy (ε = 0) for 1000 separate episodes, so evaluation does not change what the agent has learned.

4. Experimental setup

The environment is Gymnasium's FrozenLake-v1 (4x4, is_slippery=True) [1]. Everything else (random baseline, epsilon-greedy choice, SARSA, evaluation, plots) is in one file, main.py, and runs with the command python main.py (see README).

Setting	Value
Random seeds	5 independent runs: 0, 1, 2, 3, 4
Training	5000 episodes per seed
Hyperparameters	α = 0.1, γ = 0.99
Parameter study	ε = 0.1 versus ε = 0.3 (everything else fixed)
Evaluation	1000 episodes per seed, greedy policy, Q frozen, separate environment seed (seed + 100)
Baseline	Uniformly random action in every step, 1000 episodes per seed
Metric	Success rate: fraction of episodes that reach the goal
Software	Python, gymnasium, numpy, matplotlib (see requirements.txt)

Training and evaluation are separate: training performance (Figure 1) is the moving average over 100 episodes of the reward collected while exploring, and the final comparison (Figure 2 and Table 1) uses the greedy policy on fresh episodes. I chose 5000 training episodes because the training curve for ε = 0.1 flattens after roughly 3000 episodes (Figure 1); I did not tune this number further.

5. Results

SARSA with ε = 0.1 reached a success rate of 0.604 under greedy evaluation, about 40 times the random baseline (0.015).

Table 1. Success rate under greedy evaluation (1000 episodes per seed), mean ± standard deviation over 5 seeds.

Method	Success rate (mean ± std)
Random baseline	0.015 ± 0.003

Method	Success rate (mean ± std)
SARSA, ε = 0.1	0.604 ± 0.188
SARSA, ε = 0.3	0.473 ± 0.176

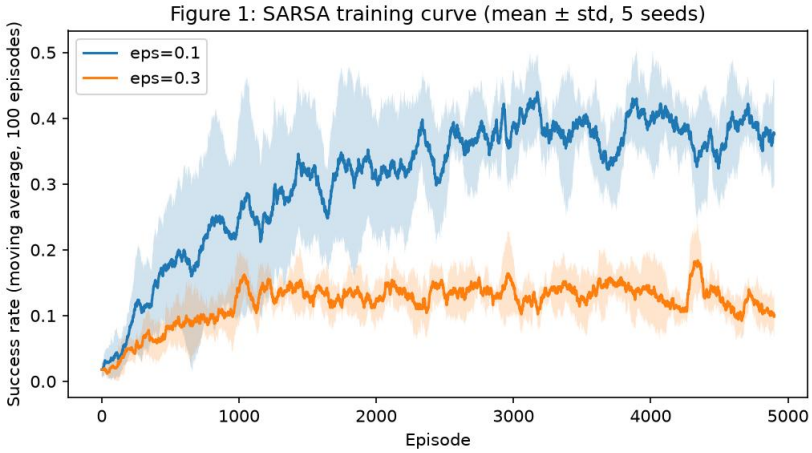


Figure 1. SARSA training curve: moving average over 100 episodes of the training success rate, mean ± std over 5 seeds, for ε = 0.1 and 0.3.

In Figure 1 the ε = 0.1 curve rises quickly during the first 1000 episodes and then levels off at a training success rate of roughly 0.35-0.40 (read from the plot). The ε = 0.3 curve stays much lower, at roughly 0.10-0.15, and is almost flat after about 1000 episodes. The shaded band for ε = 0.1 is wide, so the seeds learned at different speeds.

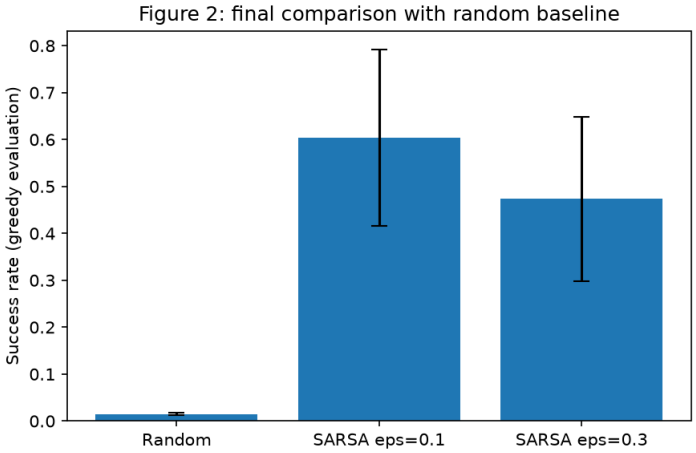


Figure 2. Final comparison: greedy-evaluation success rate of the random baseline and of SARSA with ε = 0.1 and 0.3 (mean, error bars = std over 5 seeds).

Parameter study (ε). Increasing ε from 0.1 to 0.3 lowered the training success rate clearly (Figure 1) and lowered the final greedy success rate from 0.604 to 0.473 (Table 1). The two final results differ by 0.13 while each has a standard deviation of about 0.18, so with 5 seeds this difference is suggestive but not conclusive.

Learned policies (seed 4). Greedy action in each cell (0 = left, 1 = down, 2 = right, 3 = up). Entries for holes and the goal are meaningless because Q is never updated there.

ε = 0.1	**ε = 0.3**
0 3 0 1	0 3 0 3
0 0 2 0	0 0 0 0
3 1 0 0	3 1 0 0
0 2 1 0	0 1 1 0

6. Discussion, limitations and next step

Learning works. With the same number of episodes, SARSA raised the success rate from 1.5% (random) to about 60% (ε = 0.1), so the agent extracted useful structure from interaction alone.

Training versus evaluation. Training success (about 0.35-0.40 for ε = 0.1) is lower than greedy evaluation (0.604). The training episodes include random moves, and on slippery ice a random move near a hole is costly. The evaluation uses ε = 0 and so measures the learned policy itself.

Why ε = 0.3 was worse (hypothesis, not tested). SARSA is on-policy, so its Q values describe the policy that explores 30% of the time. With that much random movement the agent falls into holes more often, which pushes the values down and slows learning. A decaying ε would test this idea.

Limitations and one unstable result.

- The standard deviation across seeds is large (about 0.18), so some seeds learned a much better policy than others. With only 5 seeds, the ranking of ε = 0.1 and 0.3 is not statistically solid.
- The learned policy is not optimal. For example, in seed 4 with ε = 0.1 the greedy action in cell 3 is down, which can lead into the hole in cell 7, while up can never end in a hole. A likely reason is that this cell is rarely visited, so its values are poorly estimated.
- ε and α were kept constant, which keeps the training curves noisy. I did not compute the optimal policy, so I cannot say how close 0.60 is to the best possible result.

With more time I would test: 20 or more seeds; a decaying ε schedule; a study of α; value iteration on the known model as an upper reference.

7. AI Use Statement

I used Claude (Anthropic) for planning the structure of the project, helping with the code in main.py.

I wrote parts of the code myself, the constants, the environment setup and the random baseline, and I typed and ran the rest step by step. I ran the code on my computer and collected all results reported here. I checked that the random baseline gives a success rate near 1.5%, as expected for this environment, and that SARSA clearly improves on it. I also chose the project structure (one main.py with functions) and the experiment design, and I edited the report text in my own words.

8. References

Gymnasium documentation, Farama Foundation, FrozenLake environment (FrozenLake-v1). https://gymnasium.farama.org/

R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, 2018 (SARSA: on-policy TD control, Chapter 6). http://incompleteideas.net/book/the-book-2nd.html

Learning record

Date	Activity	Evidence available
25 Oct 2026	**Lab 1: First-visit Monte Carlo policy evaluation on a three-state chain**	**Notebook Lab1.ipynb**
25 Oct 2026	**Sutton and Barto , Reinforcement Learning: An Introduction, 2nd edition, Chapter 3 : I readed for the MDP formulation and the choice of gamma.**	**I used it as source for project**
25 Oct 2026	**Sutton and Barto , Reinforcement Learning: An Introduction, 2nd edition, Chapter 6, Section 6.4. i readed for the update rule and the epsilon-greedy policy.**	**I used it as source for project**
26 Oct 2026	**This midterm project. Built the FrozenLake setup, random baseline, tabular SARSA, 5-seed evaluation and an epsilon study; produced the figures and wrote the report.**	**main.py, results table, Figures 1 and 2, README**

Reflection.

Before starting this project, I did not fully grasp the essence of the Monte Carlo method. In the first laboratory assignment, I ran numerous episodes and calculated the average of the results. This clearly demonstrated that the method does not require knowledge of the environment's rules. It also became evident that the cumulative reward can only be calculated once an episode has concluded. The error table shows that results were unsatisfactory when the number of episodes was low, but accuracy improved as that number increased. Through this project, I learned that the SARSA algorithm operates differently: it updates the Q-table after every step, rather than waiting for the episode to end.

One point remains somewhat unclear to me: why did a higher epsilon value lead to poorer results? I suspect it is because SARSA evaluates the value of the policy that the agent is actually following. If epsilon is large, the agent takes many random actions and more frequently falls into traps, which causes the state-value estimates to decrease. However, I have not tested this hypothesis, so I cannot be certain of its validity. If I had more time, I would conduct further experiments using different random seeds and try gradually decreasing the epsilon value. I would also revisit the section on SARSA in the book by Sutton and Barto to compare what is written there with my own conjectures.